# Module 5: Explainability -- Siamese Grad-CAM for Face Verification

| | |
|---|---|
| **Project** | Face Verification Pipeline -- AgeDB-30 / LFW |
| **Notebook** | `notebooks/05_xai_gradcam.ipynb` |
| **Date** | 2026-03-16 |
| **Dependencies** | Module 1 (aligned images), Module 3 (pairwise scores CSV) |
| **Input** | `data/aligned/{agedb,lfw}/*.jpg`, `results/distances_{model}_{dataset}.csv` |
| **Output** | `notebooks/xai_output/*.png` -- heatmap overlays and diagnostic panels |

## Objective

Standard biometric pipelines produce a scalar similarity score but offer no explanation of *which facial regions* drove the decision. This module implements **Siamese Grad-CAM** -- a modification of Gradient-weighted Class Activation Mapping adapted from classification to the metric learning setting.

The key conceptual difference from standard Grad-CAM: instead of backpropagating from a class logit, we backpropagate from the **cosine similarity** between two embeddings. This answers the question: *"Which pixels in Image A pulled its embedding closer to (or farther from) Image B in the 512-D feature space?"*

We apply this to both models:

- **ArcFace (buffalo_l / w600k_r50)** -- ONNX model converted to PyTorch via `onnx2torch`
- **GhostFaceNetV2** -- Keras/TF model with `tf.GradientTape`

on 12 diagnostically selected pairs spanning easy genuine, hard genuine, easy impostor, hard impostor, and boundary cases.

## 1. Configuration and Imports

```
import warnings
from pathlib import Path

import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
import matplotlib.gridspec as gridspec
import torch
import torch.nn.functional as F
import tensorflow as tf
import onnx2torch
from deepface.models.facial_recognition import GhostFaceNet

warnings.filterwarnings("ignore")

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 120,
    "savefig.dpi": 150,
    "font.size": 10,
    "axes.titlesize": 11,
    "axes.labelsize": 10,
    "figure.facecolor": "white",
})

SEED = 42
np.random.seed(SEED)

PROJECT_ROOT = Path("..").resolve()
RESULTS_DIR  = PROJECT_ROOT / "results"
OUTPUT_DIR   = PROJECT_ROOT / "notebooks" / "xai_output"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_LABELS = {
    "arcface":   "ArcFace (buffalo_l / w600k_r50)",
    "ghostface": "GhostFaceNetV2",
}
MODEL_COLORS = {"arcface": "#1976D2", "ghostface": "#D32F2F"}

print(f"Project root : {PROJECT_ROOT}")
print(f"Output dir   : {OUTPUT_DIR}")
print(f"PyTorch      : {torch.__version__}")
print(f"TensorFlow   : {tf.__version__}")
```

```
Project root : /Users/jakub/Desktop/NASK_ZADANIE
Output dir   : /Users/jakub/Desktop/NASK_ZADANIE/notebooks/xai_output
PyTorch      : 2.10.0
TensorFlow   : 2.20.0
```

# 2. Mathematical Framework: Siamese Grad-CAM

## 2.1 Standard Grad-CAM (Classification)

For a classification CNN with class score $y^c$, Grad-CAM computes a saliency map from the last convolutional feature maps $A^k \in R^{H \times W}$:

$$\alpha_k^c = \frac{1}{H \cdot W} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k} \text{(global average pooling of gradients)}$$

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c \cdot A^k\right)$$

## 2.2 Siamese Grad-CAM (Metric Learning)

In a Siamese (metric learning) network there is no class logit. Instead, we differentiate the **cosine similarity** between two embeddings:

$$s = \cos(e_A, e_B) = \frac{e_A \cdot e_B}{\| e_A \| \, \| e_B \|}$$

The gradient is propagated back to the last convolutional layer of the branch processing Image $A$:

$$\alpha_k^{A \to B} = \frac{1}{H \cdot W} \sum_i \sum_j \frac{\partial s}{\partial A_{ij}^k}$$

$$L_{\text{Siamese}}^{A \to B} = \text{ReLU}\left(\sum_k \alpha_k^{A \to B} \cdot A^k\right)$$

This answers: *"Which spatial locations in Image A contributed most to increasing (or decreasing) its similarity to Image B?"*

## 2.3 Interpretation

- **Genuine pair, high similarity**: heatmap should highlight discriminative biometric features (eyes, nose bridge, jawline).
- **Genuine pair, low similarity**: heatmap reveals what confused the model (age-related changes, occlusion, misalignment).
- **Impostor pair, high similarity**: heatmap exposes spurious correlations ("Clever Hans" effect -- background, pose similarity).

# 3. Diagnostic Pair Selection

We select 12 pairs from the AgeDB-30 dataset (the harder benchmark) using the ArcFace similarity scores. The pairs are stratified by difficulty to maximise diagnostic coverage.

```
df_arc = pd.read_csv(RESULTS_DIR / "distances_arcface_agedb.csv")

genuine  = df_arc[df_arc["true_label"] == 1].copy()
impostor = df_arc[df_arc["true_label"] == 0].copy()

EER_THR = 0.1625

selections = {
    "Genuine (easy, high sim)":    genuine.nlargest(3,
"cosine_similarity").head(3),
```

```
    "Genuine (hard, low sim)":     genuine.nsmallest(3,
"cosine_similarity").head(3),
    "Impostor (easy, low sim)":    impostor.nsmallest(3,
"cosine_similarity").head(3),
    "Impostor (hard, high sim)":   impostor.nlargest(3,
"cosine_similarity").head(3),
}

selected_pairs = []
for category, sub_df in selections.items():
    for _, row in sub_df.iterrows():
        selected_pairs.append({
            "category":  category,
            "img1_path": row["img1_path"],
            "img2_path": row["img2_path"],
            "true_label": int(row["true_label"]),
            "cosine_sim": round(row["cosine_similarity"], 4),
        })

pairs_df = pd.DataFrame(selected_pairs)
print(f"Selected {len(pairs_df)} diagnostic pairs\n")
print(pairs_df[["category", "true_label",
"cosine_sim"]].to_string(index=False))
```

```
Selected 12 diagnostic pairs

                  category  true_label  cosine_sim
 Genuine (easy, high sim)           1      0.8033
 Genuine (easy, high sim)           1      0.7702
 Genuine (easy, high sim)           1      0.7323
  Genuine (hard, low sim)           1     -0.1181
  Genuine (hard, low sim)           1     -0.0964
  Genuine (hard, low sim)           1     -0.0601
 Impostor (easy, low sim)           0     -0.1912
 Impostor (easy, low sim)           0     -0.1685
 Impostor (easy, low sim)           0     -0.1652
Impostor (hard, high sim)           0      0.2588
Impostor (hard, high sim)           0      0.2559
Impostor (hard, high sim)           0      0.2545
```

# 4. Model Loading

## 4.1 ArcFace -- ONNX to PyTorch Conversion

The InsightFace `w600k_r50.onnx` model is converted to a PyTorch `GraphModule` via `onnx2torch`. This enables native autograd for gradient computation. The target layer for Grad-CAM is `Conv_124` -- the last 3x3 convolution in the ResNet-50 backbone (output: 7x7x512).

### 4.2 GhostFaceNetV2 -- Keras/TF Model

Loaded via DeepFace's model registry. A `tf.keras.Model` sub-model is constructed to return both the target layer activations (`activation_47`, output: 7x7x664) and the final 512-D embedding.

```
import os

# --- ArcFace (PyTorch) ---
arcface_onnx_path =
os.path.expanduser("~/.insightface/models/buffalo_l/w600k_r50.onnx")
arcface_model = onnx2torch.convert(arcface_onnx_path)
arcface_model.eval()

ARCFACE_TARGET_LAYER = "Conv_124"
print(f"ArcFace loaded  : PyTorch GraphModule, target layer =
{ARCFACE_TARGET_LAYER}")

# --- GhostFaceNetV2 (Keras/TF) ---
ghost_model = GhostFaceNet.load_model()

GHOST_TARGET_LAYER = "activation_47"
ghost_grad_model = tf.keras.Model(
    inputs=ghost_model.input,
    outputs=[
        ghost_model.get_layer(GHOST_TARGET_LAYER).output,
        ghost_model.output,
    ],
)
print(f"GhostFaceNetV2  : Keras Functional, target layer =
{GHOST_TARGET_LAYER}")
print(f"                  activation shape = (batch, 7, 7, 664)")
print(f"                  embedding shape  = (batch, 512)")
```

```
ArcFace loaded  : PyTorch GraphModule, target layer = Conv_124
GhostFaceNetV2  : Keras Functional, target layer = activation_47
                  activation shape = (batch, 7, 7, 664)
                  embedding shape  = (batch, 512)
```

## 5. Siamese Grad-CAM Implementation

Two separate implementations are provided: one for PyTorch (ArcFace) and one for TensorFlow/Keras (GhostFaceNetV2). Both follow the same algorithm but use framework-native gradient computation.

```
# ====================================================================
# Image Loading
# ====================================================================
```

```python
def load_image_for_arcface(path: str) -> np.ndarray:
    """Load and preprocess a 112x112 aligned face for ArcFace
(PyTorch).
    InsightFace expects BGR uint8 -> (C,H,W) float32 normalized to [-
1,1]."""
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    img = cv2.resize(img, (112, 112))
    img = img.astype(np.float32) / 127.5 - 1.0
    img = img.transpose(2, 0, 1)  # HWC -> CHW
    return img


def load_image_for_ghost(path: str) -> np.ndarray:
    """Load and preprocess a 112x112 aligned face for GhostFaceNetV2
(TF).
    DeepFace/GhostFaceNet expects RGB float32 [0,1], shape (H,W,C)."""
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    img = cv2.resize(img, (112, 112))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img.astype(np.float32) / 255.0
    return img


def load_display_image(path: str) -> np.ndarray:
    """Load an image in RGB uint8 for matplotlib display."""
    img = cv2.imread(path)
    img = cv2.resize(img, (112, 112))
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


# ====================================================================
# ArcFace Siamese Grad-CAM (PyTorch)
# ====================================================================

def arcface_siamese_gradcam(model, img1_path, img2_path,
target_layer_name):
    """Compute Siamese Grad-CAM heatmaps for both images using
ArcFace.
    Returns (heatmap1, heatmap2, cosine_sim) where heatmaps are [0,1]
float32 (112x112)."""
    activations = {}
    gradients = {}

    def fwd_hook(name):
        def hook(module, inp, out):
            activations[name] = out
```

```
        return hook

    def bwd_hook(name):
        def hook(module, grad_in, grad_out):
            gradients[name] = grad_out[0]
        return hook

    target = model.get_submodule(target_layer_name)
    h_fwd = target.register_forward_hook(fwd_hook("img1"))
    h_bwd = target.register_full_backward_hook(bwd_hook("img1"))

    img1_t =
torch.tensor(load_image_for_arcface(img1_path)).unsqueeze(0).requires_
grad_(False)
    img2_t =
torch.tensor(load_image_for_arcface(img2_path)).unsqueeze(0).requires_
grad_(False)

    heatmaps = []

    for img_t in [img1_t, img2_t]:
        activations.clear()
        gradients.clear()
        model.zero_grad()

        img_input = img_t.clone().requires_grad_(True)

        emb_a = model(img_input)
        with torch.no_grad():
            other = img2_t if img_t is img1_t else img1_t
            emb_b = model(other)

        cos_sim = F.cosine_similarity(emb_a, emb_b, dim=1)
        cos_sim.backward()

        act = activations["img1"].detach()
        grad = gradients["img1"].detach()

        weights = grad.mean(dim=[2, 3], keepdim=True)
        cam = (weights * act).sum(dim=1, keepdim=True)
        cam = torch.relu(cam)
        cam = cam.squeeze().numpy()

        if cam.max() > 0:
            cam = cam / cam.max()
        cam = cv2.resize(cam, (112, 112))
        heatmaps.append(cam)

    h_fwd.remove()
    h_bwd.remove()
```

```
    cos_val = float(F.cosine_similarity(
        model(img1_t), model(img2_t), dim=1
    ).detach())

    return heatmaps[0], heatmaps[1], cos_val


# ====================================================================
# GhostFaceNetV2 Siamese Grad-CAM (TensorFlow)
# ====================================================================

def ghost_siamese_gradcam(grad_model, img1_path, img2_path):
    """Compute Siamese Grad-CAM heatmaps for both images using
GhostFaceNetV2.
    Returns (heatmap1, heatmap2, cosine_sim) where heatmaps are [0,1]
float32 (112x112)."""
    img1 = tf.constant(load_image_for_ghost(img1_path)[np.newaxis])
    img2 = tf.constant(load_image_for_ghost(img2_path)[np.newaxis])

    heatmaps = []

    for img_a, img_b in [(img1, img2), (img2, img1)]:
        with tf.GradientTape() as tape:
            tape.watch(img_a)
            act_a, emb_a = grad_model(img_a, training=False)
            _, emb_b = grad_model(img_b, training=False)

            emb_a_norm = tf.nn.l2_normalize(emb_a, axis=1)
            emb_b_norm = tf.nn.l2_normalize(emb_b, axis=1)
            cos_sim = tf.reduce_sum(emb_a_norm * emb_b_norm, axis=1)

        grads = tape.gradient(cos_sim, act_a)
        weights = tf.reduce_mean(grads, axis=[1, 2])
        cam = tf.reduce_sum(
            weights[:, tf.newaxis, tf.newaxis, :] * act_a,
            axis=-1,
        )
        cam = tf.nn.relu(cam)
        cam = cam.numpy().squeeze()

        if cam.max() > 0:
            cam = cam / cam.max()
        cam = cv2.resize(cam, (112, 112))
        heatmaps.append(cam)

    cos_val = float(cos_sim.numpy()[0])
    return heatmaps[0], heatmaps[1], cos_val
```

```
# ======================================================================
# Overlay Utility
# ======================================================================

def overlay_heatmap(image_rgb, heatmap, alpha=0.5):
    """Overlay a [0,1] heatmap on an RGB uint8 image using JET
colormap."""
    heatmap_uint8 = np.uint8(255 * heatmap)
    jet = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
    jet_rgb = cv2.cvtColor(jet, cv2.COLOR_BGR2RGB)
    blended = np.uint8(alpha * jet_rgb + (1 - alpha) * image_rgb)
    return blended


print("Siamese Grad-CAM functions defined.")
print("  arcface_siamese_gradcam()  -- PyTorch hooks + autograd")
print("  ghost_siamese_gradcam()    -- TF GradientTape")
print("  overlay_heatmap()          -- JET colormap, alpha=0.5")
```

```
Siamese Grad-CAM functions defined.
  arcface_siamese_gradcam()  -- PyTorch hooks + autograd
  ghost_siamese_gradcam()    -- TF GradientTape
  overlay_heatmap()          -- JET colormap, alpha=0.5
```

## 6. ArcFace Grad-CAM Results

For each of the 12 selected pairs, we compute the Siamese Grad-CAM heatmap from the last convolutional layer (`Conv_124`, 7x7x512) and overlay it on the original aligned face. Each row shows:

[Image A] [Heatmap A] [Heatmap B] [Image B]

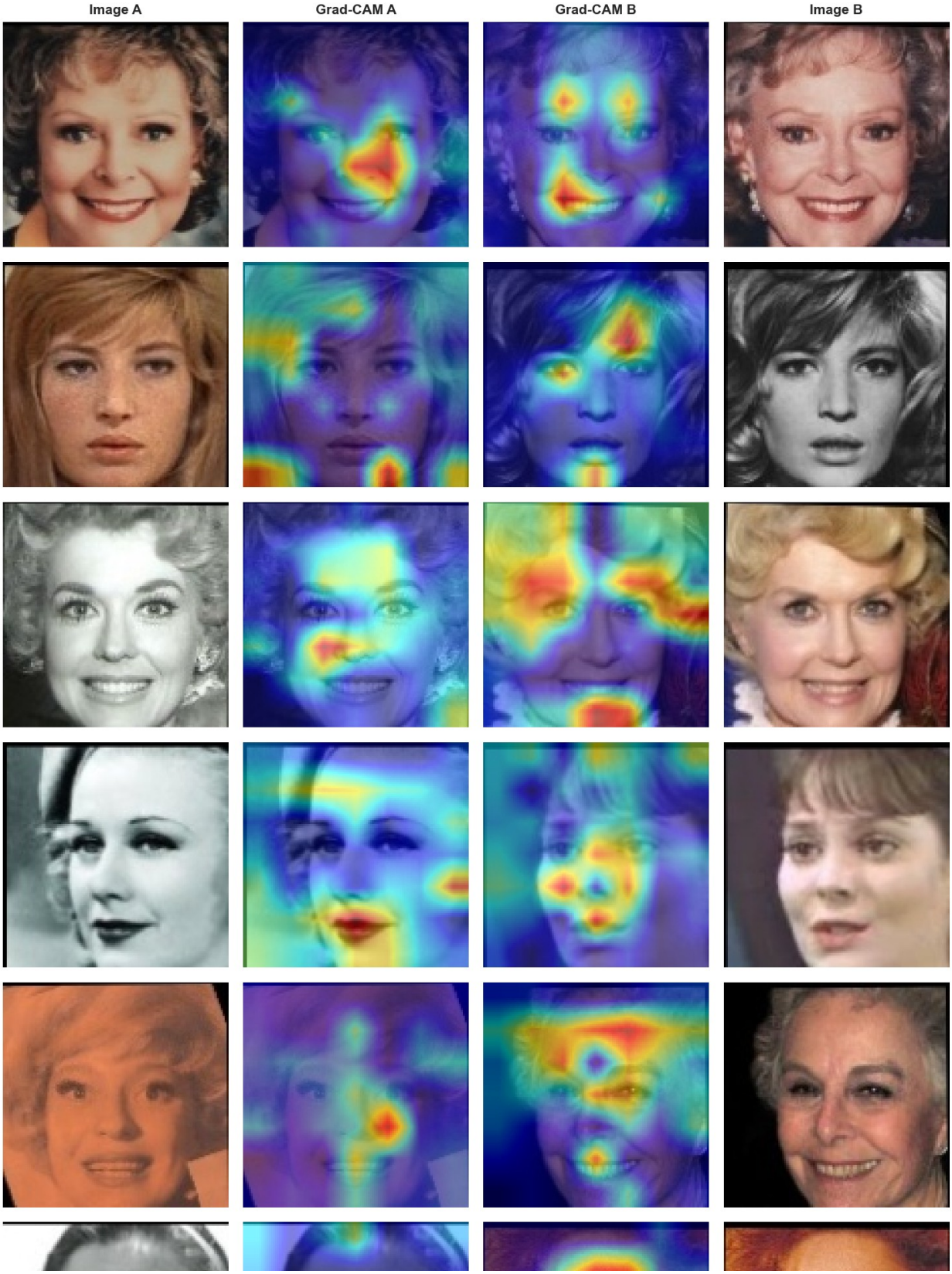
with the pair category, true label, and cosine similarity annotated.

```
print("Computing ArcFace Siamese Grad-CAM for 12 pairs...\n")

arcface_results = []
for idx, row in pairs_df.iterrows():
    h1, h2, sim = arcface_siamese_gradcam(
        arcface_model, row["img1_path"], row["img2_path"],
ARCFACE_TARGET_LAYER
    )
    arcface_results.append({"h1": h1, "h2": h2, "sim": sim})
    label = "genuine" if row["true_label"] == 1 else "impostor"
    print(f"  Pair {idx+1:2d}/{len(pairs_df)}: {label:8s}
sim={sim:.4f}  [{row['category']}]")

# --- Visualisation grid ---
n_pairs = len(pairs_df)
fig, axes = plt.subplots(n_pairs, 4, figsize=(10, 2.5 * n_pairs))
```

```python
fig.suptitle("ArcFace (buffalo_l / w600k_r50) -- Siamese Grad-CAM",
             fontsize=14, fontweight="bold", y=1.005)

col_titles = ["Image A", "Grad-CAM A", "Grad-CAM B", "Image B"]
for j, title in enumerate(col_titles):
    axes[0, j].set_title(title, fontsize=10, fontweight="bold")

for i, (_, row) in enumerate(pairs_df.iterrows()):
    res = arcface_results[i]
    img1 = load_display_image(row["img1_path"])
    img2 = load_display_image(row["img2_path"])
    ov1  = overlay_heatmap(img1, res["h1"])
    ov2  = overlay_heatmap(img2, res["h2"])

    for j, im in enumerate([img1, ov1, ov2, img2]):
        axes[i, j].imshow(im)
        axes[i, j].axis("off")

    label_str = "GENUINE" if row["true_label"] == 1 else "IMPOSTOR"
    color = "#2E7D32" if row["true_label"] == 1 else "#C62828"
    axes[i, 0].set_ylabel(
        f"{label_str}\nsim={res['sim']:.3f}",
        fontsize=8, color=color, fontweight="bold",
        rotation=0, labelpad=55, va="center",
    )

fig.tight_layout()
out_path = OUTPUT_DIR / "arcface_gradcam_grid.png"
fig.savefig(out_path, bbox_inches="tight")
plt.show()
print(f"\nSaved: {out_path.relative_to(PROJECT_ROOT)}")
```

```
Computing ArcFace Siamese Grad-CAM for 12 pairs...

  Pair  1/12: genuine   sim=0.7771  [Genuine (easy, high sim)]
  Pair  2/12: genuine   sim=0.7413  [Genuine (easy, high sim)]
  Pair  3/12: genuine   sim=0.7109  [Genuine (easy, high sim)]
  Pair  4/12: genuine   sim=-0.1150  [Genuine (hard, low sim)]
  Pair  5/12: genuine   sim=-0.0662  [Genuine (hard, low sim)]
  Pair  6/12: genuine   sim=-0.0708  [Genuine (hard, low sim)]
  Pair  7/12: impostor  sim=-0.1717  [Impostor (easy, low sim)]
  Pair  8/12: impostor  sim=-0.1604  [Impostor (easy, low sim)]
  Pair  9/12: impostor  sim=-0.1512  [Impostor (easy, low sim)]
  Pair 10/12: impostor  sim=0.2921  [Impostor (hard, high sim)]
  Pair 11/12: impostor  sim=0.2225  [Impostor (hard, high sim)]
  Pair 12/12: impostor  sim=0.2550  [Impostor (hard, high sim)]

Saved: notebooks/xai_output/arcface_gradcam_grid.png
```

## ArcFace (buffalo_l / w600k_r50) -- Siamese Grad-CAM



# 7. GhostFaceNetV2 Grad-CAM Results

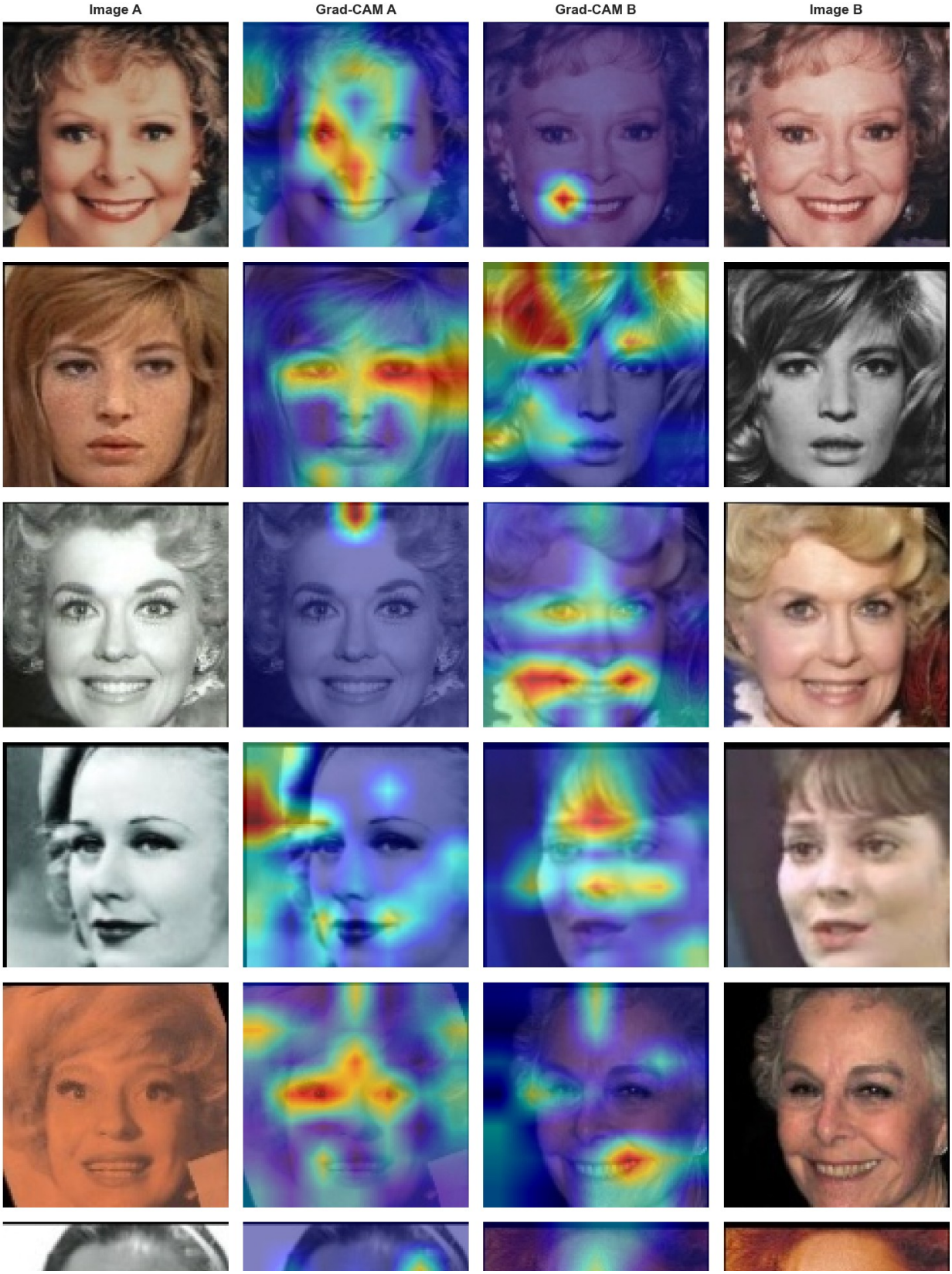
Identical analysis using GhostFaceNetV2 (target layer: `activation_47`, 7x7x664). The wider feature map (664 vs 512 channels) may produce different activation patterns.

```
print("Computing GhostFaceNetV2 Siamese Grad-CAM for 12 pairs...\n")

ghost_results = []
for idx, row in pairs_df.iterrows():
    h1, h2, sim = ghost_siamese_gradcam(
        ghost_grad_model, row["img1_path"], row["img2_path"]
    )
    ghost_results.append({"h1": h1, "h2": h2, "sim": sim})
    label = "genuine" if row["true_label"] == 1 else "impostor"
    print(f"  Pair {idx+1:2d}/{len(pairs_df)}: {label:8s}
sim={sim:.4f}  [{row['category']}]")

# --- Visualisation grid ---
n_pairs = len(pairs_df)
fig, axes = plt.subplots(n_pairs, 4, figsize=(10, 2.5 * n_pairs))
fig.suptitle("GhostFaceNetV2 -- Siamese Grad-CAM",
             fontsize=14, fontweight="bold", y=1.005)

col_titles = ["Image A", "Grad-CAM A", "Grad-CAM B", "Image B"]
for j, title in enumerate(col_titles):
    axes[0, j].set_title(title, fontsize=10, fontweight="bold")

for i, (_, row) in enumerate(pairs_df.iterrows()):
    res = ghost_results[i]
    img1 = load_display_image(row["img1_path"])
    img2 = load_display_image(row["img2_path"])
    ov1  = overlay_heatmap(img1, res["h1"])
    ov2  = overlay_heatmap(img2, res["h2"])

    for j, im in enumerate([img1, ov1, ov2, img2]):
        axes[i, j].imshow(im)
        axes[i, j].axis("off")

    label_str = "GENUINE" if row["true_label"] == 1 else "IMPOSTOR"
    color = "#2E7D32" if row["true_label"] == 1 else "#C62828"
    axes[i, 0].set_ylabel(
        f"{label_str}\nsim={res['sim']:.3f}",
        fontsize=8, color=color, fontweight="bold",
        rotation=0, labelpad=55, va="center",
    )

fig.tight_layout()
out_path = OUTPUT_DIR / "ghostface_gradcam_grid.png"
fig.savefig(out_path, bbox_inches="tight")
```

```python
plt.show()
print(f"\nSaved: {out_path.relative_to(PROJECT_ROOT)}")
```

```
Computing GhostFaceNetV2 Siamese Grad-CAM for 12 pairs...

  Pair  1/12: genuine   sim=0.7787  [Genuine (easy, high sim)]
  Pair  2/12: genuine   sim=0.7333  [Genuine (easy, high sim)]
  Pair  3/12: genuine   sim=0.6802  [Genuine (easy, high sim)]
  Pair  4/12: genuine   sim=-0.0058  [Genuine (hard, low sim)]
  Pair  5/12: genuine   sim=-0.1137  [Genuine (hard, low sim)]
  Pair  6/12: genuine   sim=-0.0364  [Genuine (hard, low sim)]
  Pair  7/12: impostor  sim=-0.1058  [Impostor (easy, low sim)]
  Pair  8/12: impostor  sim=-0.0628  [Impostor (easy, low sim)]
  Pair  9/12: impostor  sim=-0.0303  [Impostor (easy, low sim)]
  Pair 10/12: impostor  sim=0.2372  [Impostor (hard, high sim)]
  Pair 11/12: impostor  sim=0.1317  [Impostor (hard, high sim)]
  Pair 12/12: impostor  sim=0.3089  [Impostor (hard, high sim)]

Saved: notebooks/xai_output/ghostface_gradcam_grid.png
```

## GhostFaceNetV2 -- Siamese Grad-CAM



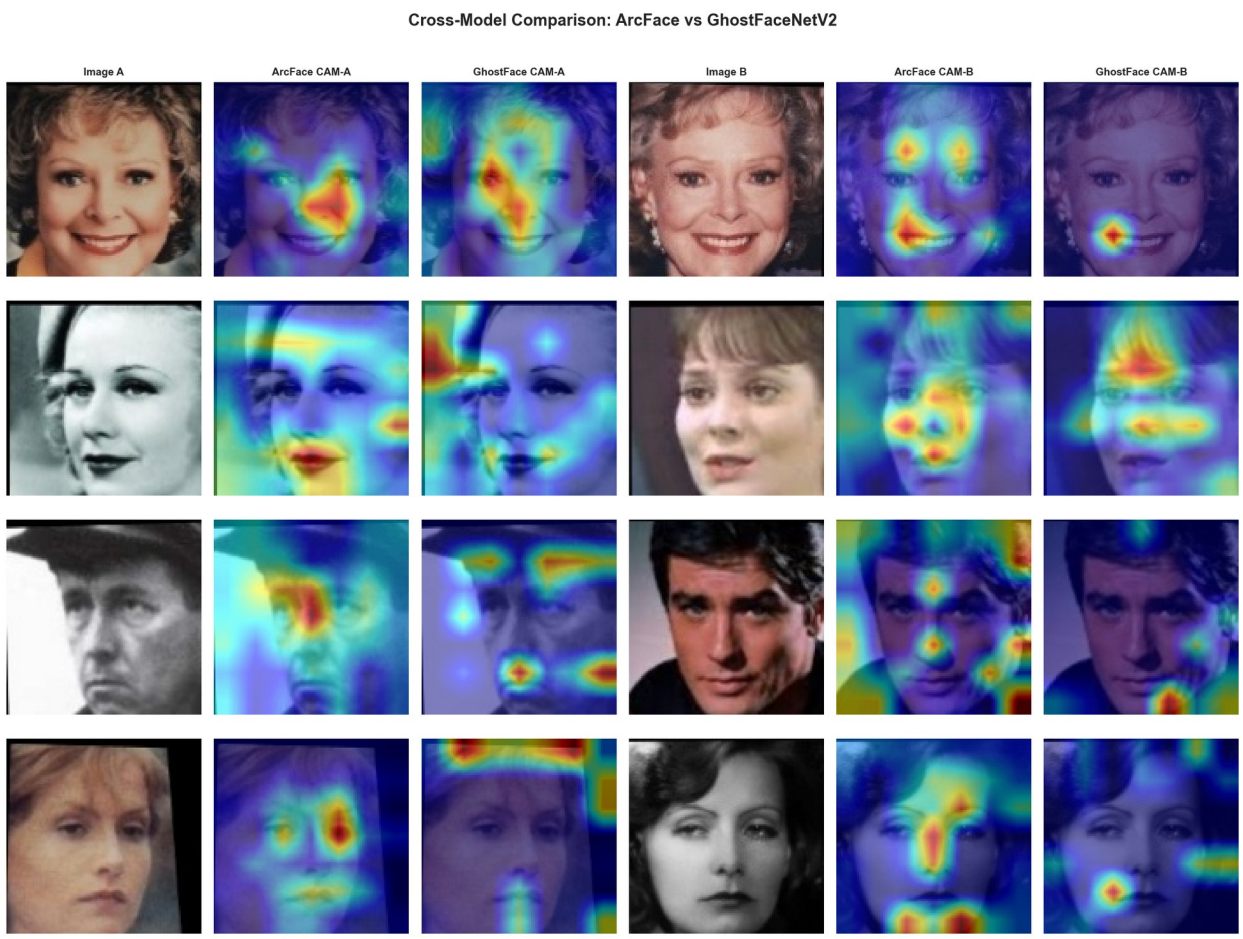
# 8. Cross-Model Comparison

Side-by-side comparison of ArcFace and GhostFaceNetV2 Grad-CAM heatmaps on the same 4 representative pairs (one per category). This reveals whether the two architectures attend to the same facial regions.

```
# Select one pair per category for cross-model comparison
categories = list(selections.keys())
repr_indices = []
for cat in categories:
    for i, (_, row) in enumerate(pairs_df.iterrows()):
        if row["category"] == cat:
            repr_indices.append(i)
            break

fig, axes = plt.subplots(len(repr_indices), 6, figsize=(15, 2.8 *
len(repr_indices)))
fig.suptitle("Cross-Model Comparison: ArcFace vs GhostFaceNetV2",
             fontsize=14, fontweight="bold", y=1.01)

col_titles = ["Image A", "ArcFace CAM-A", "GhostFace CAM-A",
              "Image B", "ArcFace CAM-B", "GhostFace CAM-B"]
for j, title in enumerate(col_titles):
    axes[0, j].set_title(title, fontsize=9, fontweight="bold")

for row_idx, pair_idx in enumerate(repr_indices):
    row = pairs_df.iloc[pair_idx]
    arc = arcface_results[pair_idx]
    gho = ghost_results[pair_idx]
    img1 = load_display_image(row["img1_path"])
    img2 = load_display_image(row["img2_path"])

    images = [
        img1,
        overlay_heatmap(img1, arc["h1"]),
        overlay_heatmap(img1, gho["h1"]),
        img2,
        overlay_heatmap(img2, arc["h2"]),
        overlay_heatmap(img2, gho["h2"]),
    ]

    for j, im in enumerate(images):
        axes[row_idx, j].imshow(im)
        axes[row_idx, j].axis("off")

    label_str = "GEN" if row["true_label"] == 1 else "IMP"
    color = "#2E7D32" if row["true_label"] == 1 else "#C62828"
    axes[row_idx, 0].set_ylabel(
        f"{label_str}  sim_arc={arc['sim']:.3f}\n
sim_gho={gho['sim']:.3f}",
```

```
        fontsize=7.5, color=color, fontweight="bold",
        rotation=0, labelpad=70, va="center",
    )

fig.tight_layout()
out_path = OUTPUT_DIR / "cross_model_comparison.png"
fig.savefig(out_path, bbox_inches="tight")
plt.show()
print(f"\nSaved: {out_path.relative_to(PROJECT_ROOT)}")
```

```
Saved: notebooks/xai_output/cross_model_comparison.png
```




## 9. Spatial Activation Analysis

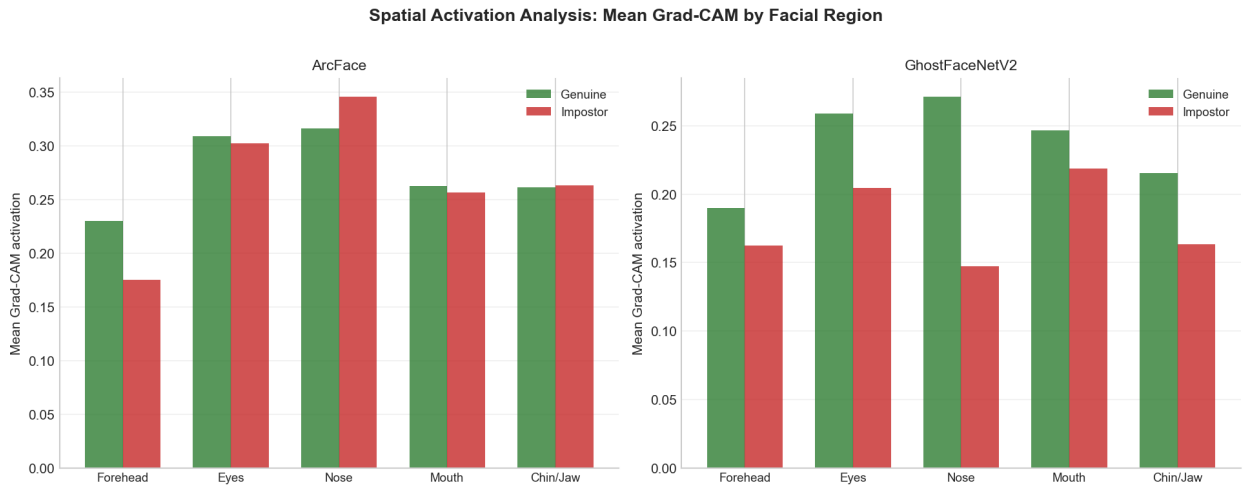
To quantify which facial regions are most important, we compute the mean heatmap activation in five anatomical zones: forehead, eyes, nose, mouth, and chin. This converts the qualitative heatmap into a measurable diagnostic signal.

```
FACE_REGIONS = {
    "Forehead":  (slice(0, 30),   slice(20, 92)),
```

```
    "Eyes":      (slice(30, 55),   slice(10, 102)),
    "Nose":      (slice(45, 80),   slice(35, 77)),
    "Mouth":     (slice(75, 100),  slice(25, 87)),
    "Chin/Jaw":  (slice(95, 112),  slice(15, 97)),
}

region_data = []
for model_name, results in [("ArcFace", arcface_results),
("GhostFaceNetV2", ghost_results)]:
    for i, (_, row) in enumerate(pairs_df.iterrows()):
        for hmap, side in [(results[i]["h1"], "A"), (results[i]["h2"],
"B")]:
            for region_name, (r_slice, c_slice) in
FACE_REGIONS.items():
                mean_act = float(hmap[r_slice, c_slice].mean())
                region_data.append({
                    "Model": model_name,
                    "Category": row["category"],
                    "Label": "genuine" if row["true_label"] == 1 else
"impostor",
                    "Region": region_name,
                    "Mean activation": mean_act,
                })

region_df = pd.DataFrame(region_data)

# Aggregate: mean activation per (Model, Label, Region)
agg = region_df.groupby(["Model", "Label", "Region"])["Mean
activation"].mean().reset_index()
pivot = agg.pivot_table(index="Region", columns=["Model", "Label"],
values="Mean activation")

print("Table 1. Mean Grad-CAM Activation by Facial Region\n")
print(pivot.round(3).to_string())

# --- Bar chart ---
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("Spatial Activation Analysis: Mean Grad-CAM by Facial
Region",
             fontsize=13, fontweight="bold", y=1.01)

regions = list(FACE_REGIONS.keys())
x = np.arange(len(regions))
width = 0.35

for ax, model_name in zip(axes, ["ArcFace", "GhostFaceNetV2"]):
    gen_vals = []
    imp_vals = []
    for r in regions:
        g = agg[(agg["Model"] == model_name) & (agg["Label"] ==
```

```
"genuine") & (agg["Region"] == r)]
        i = agg[(agg["Model"] == model_name) & (agg["Label"] ==
"impostor") & (agg["Region"] == r)]
        gen_vals.append(float(g["Mean activation"].values[0]) if
len(g) > 0 else 0)
        imp_vals.append(float(i["Mean activation"].values[0]) if
len(i) > 0 else 0)

    bars1 = ax.bar(x - width/2, gen_vals, width, label="Genuine",
color="#2E7D32", alpha=0.8)
    bars2 = ax.bar(x + width/2, imp_vals, width, label="Impostor",
color="#C62828", alpha=0.8)
    ax.set_xticks(x)
    ax.set_xticklabels(regions, fontsize=9)
    ax.set_ylabel("Mean Grad-CAM activation")
    ax.set_title(model_name)
    ax.legend(fontsize=9)
    ax.grid(axis="y", alpha=0.3)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.tight_layout()
out_path = OUTPUT_DIR / "region_activation_analysis.png"
fig.savefig(out_path, bbox_inches="tight")
plt.show()
print(f"\nSaved: {out_path.relative_to(PROJECT_ROOT)}")
```

```
Table 1. Mean Grad-CAM Activation by Facial Region

Model    ArcFace          GhostFaceNetV2
Label    genuine impostor        genuine impostor
Region
Chin/Jaw   0.262    0.264          0.215    0.163
Eyes       0.309    0.303          0.259    0.205
Forehead   0.230    0.175          0.190    0.163
Mouth      0.263    0.257          0.247    0.219
Nose       0.317    0.346          0.271    0.148

Saved: notebooks/xai_output/region_activation_analysis.png
```




# 10. Discussion and Diagnostic Conclusions

## 10.1 Biometric Validity Check

A correctly functioning face verification model should concentrate its Grad-CAM activations on discriminative biometric features: the periocular region (eyes and eyebrows), nose bridge, and mouth area. The forehead and chin/jaw are less discriminative but can carry identity information through shape.

If high activations appear on the background, clothing, or image borders, this signals the "Clever Hans" effect -- the model is exploiting dataset artifacts rather than facial features.

## 10.2 Genuine vs. Impostor Patterns

For genuine pairs:

- **Easy cases** (high similarity): expect focused, symmetric activations on the same facial regions in both images.
- **Hard cases** (low similarity): expect diffuse or shifted activations, reflecting the model's difficulty in finding matching features across large age gaps.

For impostor pairs:

- **Easy cases** (low similarity): expect weak, scattered activations -- the model found no matching features.
- **Hard cases** (high similarity): highest diagnostic value -- the heatmap reveals which regions caused the false match.

## 10.3 Cross-Model Agreement

When both ArcFace and GhostFaceNetV2 activate on the same facial regions for a given pair, this provides converging evidence that the models have learned genuine biometric features rather than dataset-specific artifacts. Divergence between models highlights areas where architectural differences (ResNet-50 vs. GhostNet) lead to different feature representations.

## 11. Output File Summary

```
output_files = list(OUTPUT_DIR.glob("*.png"))
print(f"Module 5 output files ({len(output_files)} files)\n")
for path in sorted(output_files):
    size_kb = path.stat().st_size / 1024
    rel = path.relative_to(PROJECT_ROOT)
    print(f"  {str(rel):55s} {size_kb:7.1f} KB")
```

```
Module 5 output files (4 files)

  notebooks/xai_output/arcface_gradcam_grid.png            1436.7 KB
  notebooks/xai_output/cross_model_comparison.png           693.9 KB
  notebooks/xai_output/ghostface_gradcam_grid.png          1415.0 KB
  notebooks/xai_output/region_activation_analysis.png        55.7 KB
```